Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 9

Kalibracija kamere

Neke kamere prave veliku distorziju prilikom slikanja. Dva generalna tipa distorzije su radijalna i tangencijalna distorzija. Radijalna distorzija utiče na sliku tako što se prave linije čine zakrivljenim - distorzija postaje veća što su pikseli dalje od centra slike. Tangencijalna distorzija se javlja zato što sočivo kamere nije savršeno paralelno sa ravni u kojoj je slika. U tom slučaju, neka područja na slici izgledaju bliža nego očekivano. Za više detalja možete pogledati predavanje 10.

U suštini, potrebno je da izračunamo dvije matrice kako bismo negirali efekat distorzije na slici - jedna je matrica distorzionih koeficijenata, a druga je matrica kamere. Kalibracija kamere predstavlja određivanje ove dvije matrice. Kalkulacija parametara se radi preko osnovnih geometrijskih jednačina. Jednačine zavise od odabranih kalibracijskih objekata. Mi ćemo na ovom tutorijalu naučiti kako da koristimo šahovsku tablu za kalibraciju kamere. Potrebno je uslikati šahovsku tablu kamerom minimalno dva puta kako bismo mogli računati potrebne parametre unutar OpenCV-a. Mi nećemo koristiti samo dvije slike, nego više njih, kako bismo imali što bolje rezultate.

Prvo je potrebno da pronađemo vrhove šahovske table na slici. To možemo uraditi koristeći funkciju `bool cv::findChessboardCorners(InputArray slika, Size velicina, OutputArray vrhovi, int flag = cv::CALIB_CB_ADAPTIVE_THRESH+CALIB_CB_NORMALIZE_IMAGE)`. Prvi parametar predstavlja sliku šahovske table (mora biti 8-bitna *grayscale slika* ili slika u boji), drugi parametar predstavlja broj unutrašnjih vrhova po redu i koloni table, treći parametar je izlazni niz detektovanih vrhova, i posljednji parametar predstavlja različite operacijske *flag*-ove koji mogu biti nula ili kombinacija nekih od datih vrijednosti. Nakon toga bi trebalo iscrtati na originalnoj slici gdje su pronađeni vrhovi. Zato što je ovo malo duža procedura koja se treba izvršiti nad svakom slikom koju smo uslikali, bilo bi pametno napraviti funkciju koja će da radi sve što je navedeno.

```
1   void PronadjiVrhoveTable(cv::Mat &slika_table, std::vector<cv::Point2f>
        &vrhovi) {
2
3       // Ovaj parametar predstavlja unutrasnje vrhove table, po redovima
            i kolonama:
4       cv::Size broj_vrhova(x, y);
5
6       // Funkcija za pronalazak vrhova:
7       bool pronadjeno = cv::findChessboardCorners(slika_table,
            broj_vrhova, vrhovi);
8
9       // Ako su uspjesno pronadjeni vrhovi, crtamo vrhove table:
10      if(pronadjeno)
11          cv::drawChessboardCorners(slika_table, broj_vrhova,
                cv::Mat(vrhovi), pronadjeno);
12
13      // Sada koristimo instancu klase TermCriteria:
```

```
14          cv::TermCriteria
               termcrit(cv::TermCriteria::COUNT|cv::TermCriteria::EPS, 20,
               0.03);
15          // Prethodnu varijablu iskoristimo u funkciji cv::cornerSubPix kako
               bismo nasli vise egzaktne pozicije vrhova:
16          cv::cornerSubPix(slika_table, vrhovi, cv::Size(11, 11),
               cv::Size(-1, -1), termcrit);
17
18      }
```

Prethodna funkcija predstavlja način kako se može implementirati pronalazak vrhova na šahovskoj tabli. Primijetimo da u liniji 4 varijabla `broj_vrhova` je inicijalizirana na `x, y` - objasnili smo ranije da ovaj parametar predstavlja broj unutrašnjih vrhova po redu i koloni šahovske table, dakle, potrebno ga je mijenjati na osnovu slika šahovske table koje imate. Možete izmijeniti prethodnu funkciju tako da se svaki put šalju parametri `(x, y)`, a možete samo staviti da su to brojevi unutar funkcije ukoliko znate da ste slikali istu šahovsku tablu više puta iz više uglova.

Primijetimo pored toga da su parametri koje prima naša funkcija `PronadjiVrhoveTable` poslani kao reference - to je urađeno da se smanji količina kopiranja podataka iz `main` funkcije, a pored štednje memorije štedimo i na brzini u tom slučaju. Prvi parametar koji prima naša funkcija je ustvari slika šahovske table, dok je drugi parametar vektor tačaka u koji ćemo upisati pronađene vrhove.

Dalje, u liniji 7 koristimo gotovu funkciju iz OpenCV-a o kojoj smo maloprije pričali kako bismo našli vrhove table, `cv::findChessboardCorners`. Ukoliko su uspješno pronađeni vrhovi (linija 10), onda crtamo vrhove preko gotove funkcije `cv::drawChessboardCorners` kojoj šaljemo sliku šahovske table, varijablu koja predstavlja broj unutrašnjih vrhova table po redu i po koloni, pronađeni vektor tačaka vrhova iz prethodne funkcije koji smo automatskom pretvorbom pretvorili u `cv::Mat` tip i na kraju `bool` koji govori je li uspješno prošao pronalazak vrhova. Sada je slika šahovske table izmijenjena i sadrži kružiće na svim pronađenim vrhovima.

`cv::findChessboardCorners` daje dovoljno dobre rezultate za ljudsko oko, no ona ćoškove samo aproksimativno nalazi. Pošto je cilj kalibracija kamere potrebno je da lokacije ovih vrhova budu što preciznije određene.

Iz tog razloga, koristimo sljedeću funkciju `cv::cornerSubPix` koja nam omogućava da nađemo *sub-pixel* tačnu lokaciju vrhova, u ovom slučaju, šahovske table. Dakle, ova funkcija poboljšava lokacije vrhova. Da bismo je koristili, potrebna nam je instanca klase `cv::TermCriteria` koja u svom konstruktoru prima tri parametra: tip terminacijskog kriterija, maksimalan broj iteracija ili maksimalan broj elemenata za računanje i epsilon tj. željenu preciznost pri kojoj iterativni algoritam se zaustavlja.

U našoj funkciji smo stavili da je prvi tip parametra dva navedene `enum` tipa sa *bitwise or* operatorom (možete detaljnije pronaći šta znače ova dva `enum` tipa na linku). Za drugi parametar smo stavili da algoritam `cv::cornerSubPix` zaustavi algoritam nakon 20 iteracija, a za treći parametar da se zaustavi nakon što promjena epsilon bude manja ili jednaka 0.03.

Nakon toga, pozivamo funkciju `cv::cornerSubPix` kojoj prosljeđujemo sliku šahovske table, niz do sada pronađenih vrhova u koji će se upisati preciznije lokacije vrhova, pola dužine prozora koji pretražuje sliku (u ovom slučaju je `cv::Size(11, 11)` što znači da se koristi prozor veličine 23×23), pola veličine mrtve zone u sredini zone pretraživanja gdje se sumacija u formuli za pretraživanje ne radi (kod nas je taj parmetar jednak `cv::Size(-1, -1)`, što znači da ne postoji takva veličina u našem slučaju) i na kraju kriterij koji smo prethodno inicijalizirali i objasnili.

Nakon cijele ove procedure imamo lokaciju vrhova na šahovskoj tabli! Ali to nije dovoljno za kalibraciju kamere - pogledajmo funkciju `cv::calibrateCamera` koju trebamo iskoristiti da bismo izračunali matricu kamere, koeficijente distance, rotacijski vektor i translacijski vektor - sve informacije koje su nam potrebne za kalibraciju.

Funkcija `cv::calibrateCamera` koju ćemo mi koristiti je preklopljena funkcija članica koja prima 9 parametara (originalna prima 12). Razlog zašto koristimo preklopljenu funkciju članicu je što mi nemamo sve koeficijente za napredno kalibrisanje kamere, koje originalna funkcija traži. Prototip ove funkcije glasi:

```
double cv::calibrateCamera(InputArrayOfArrays tacke_objekta,
    InputArrayOfArrays tacke_slike, Size velicina_slike,
    InputOutputArray matrica_kamere, InputOutputArray
    koeficijenti_distance, OutputArrayOfArrays rotacijski_vektor,
    OutputArrayOfArrays translacijski_vektor, int flag = 0, TermCriteria
    kriterij = TermCriteria(TermCriteria::COUNT+TermCriteria::EPS, 30,
    DBL_EPSILON))
```

Kao što možemo vidjeti, ovo je stvarno velika funkcija. Sve parametre imamo, osim parametra `tacke_objekta`. Taj parametar će kod nas biti tipa `std::vector<std::vector<cv::Point3f>>` i predstavlja vektor vektora tačaka kalibracijskog šablona u koordinacijskom prostoru kalibracijskog obrasca. Šta uopšte ovo znači za nas? Pojednostavljeno, veliki vektor ima broj elemnata onoliko koliko ima slika koje koristimo za kalibraciju i u našem slučaju svaki taj vektor je isti! U detalje kad i zašto ne moraju biti ovi vektori isti nećemo ulaziti. Razlog zašto su ovi elementi isti je što će kod nas set slika za kalibraciju (koji ćete dobiti na laboratorijskoj vježbi) uvijek sadržavati šahovsku tablu bez opstrukcija ili zaklanjanja, dakle tabla je uvijek u potpunosti vidljiva na slici. Način na koji ćemo inicijalizirati ovaj vektor je sljedeći:

```
for(int k = 0; k < broj_slika_za_kalibraciju; k++)
    for(int i = 0; i < y; i++)
        for(int j = 0; j < x; j++)
            tacke_objekta.at(k).push_back(cv::Point3f(j, i, 0));
```

Sjetimo se šta parametri `(x, y)` znače - broj unutrašnjih ivica šahovske table po redu i po koloni.

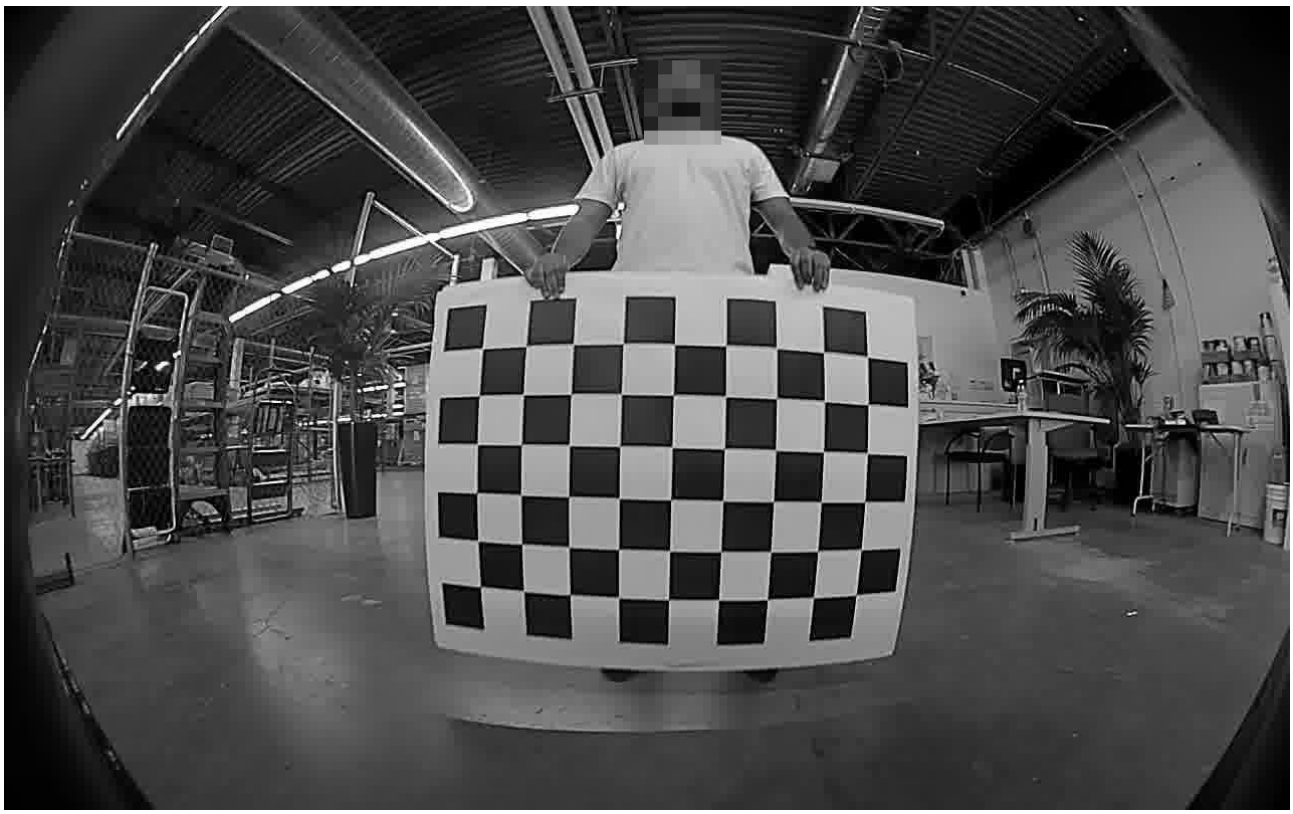
Dalje, parametar `tacke_slike` je tipa `std::vector<std::vector<cv::Point2f>>` i predstavlja sve ivice koje smo našli prethodno koristeći funkciju `PronadjiVrhoveTable`. Veliki vektor u ovom slučaju sadrži vektore ivica šahovskih ploča sa svih slika iz seta za kalibraciju.

Parametar `velicina_slike` je tipa `cv::Size` i dovoljno je uzeti jednu sliku iz seta za kalibraciju, učitati je u objekat tipa `cv::Mat` i inicijalizirati ovaj parametar kao `cv::Size(slika.rows, slika.cols)`, gdje je `slika` spomenuta učitana slika.

Parametri `matrica_kamere`, `koeficijenti_distance`, `rotacijski_vektor` i `translacijski_vektor` su svi tipa `cv::Mat`. Njih ne moramo uopšte inicijalizirati jer će njihova vrijednost biti izračunata pozivom funkcije `cv::calibrateCamera`. Posljednja dva parametra ne moramo uopšte slati jer su oni preklopljeni.

Nakon cijele ove procedure, dobili smo tražene matrice za kalibraciju kamere. Ukoliko želimo da prikažemo sada sliku koja nema distorzije na sebi, možemo koristiti funkciju `cv::undistort` koja prima sljedeće parametre: izvornu sliku koja ima distorzije u sebi, sliku u koju ćemo sačuvati rezultat funkcije nakon uklanjanja distorzije, prethodno izračunatu matricu `matrica_kamere` i koeficijente distance `koeficijenti_distance`.

Da bismo pokazali kako radi `cv::undistort`, na slici 1 je prikazana originalna slika, dok je na slici 2 prikazana slika sa uklonjenom distorzijom. Kao što možemo vidjeti, ukanjanje distorzije nije savršeno prošlo, ali svakako na slici 2 šahovska tabla puno bolje izgleda i vidno ima puno manje distorzije od slike 1, ali pošto smo kalibrisali kameru na osnovu šahovske table, vidimo da pozadina na slici 2 je iskrivljena.



Slika 1: Originalna slika



Slika 2: Slika nakon primjenjivanja `cv::undistort` funkcije

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.